

Serverless Architecture - One Page Interview Prep

Main Answer

"I deployed the AI Interview Platform as a serverless Next.js app on AWS using SST. SST is my infrastructure and deployment layer. In `sst.config.ts`, I define the app name, production protection/retention, AWS as the provider, Node.js 20 as the Lambda runtime, and server-side environment variables like `DATABASE_URL`, Upstash Redis credentials, and `GEMINI_API_KEY`."

At runtime, **CloudFront** is the public entry point and CDN. It serves cached/static assets quickly and routes dynamic requests to **Lambda**. **S3** stores static Next.js build assets like JS, CSS, and images. **Lambda** runs the dynamic Next.js/API code: session start, question fetch, answer submission, history/meta, Prisma DB calls, Redis quota/cache/locks, and Gemini calls.

I chose serverless because the app has bursty request/response traffic. I avoid managing EC2 servers, patching OSes, manual load balancing, and paying for idle compute. Lambda scales per request, S3/CloudFront serve static files efficiently, and SST gives repeatable infrastructure-as-code plus CI/CD-friendly deployments.

What Each Component Does

- **SST:** IaC/deployment framework. Provisions AWS resources, handles stages, passes environment variables, deploys with `npx sst deploy --stage production`.
- **OpenNext / SST Nextjs:** Adapts the Next.js app to AWS by separating static assets from server execution.
- **Lambda:** Stateless compute for API routes, SSR/server behavior, Prisma calls, Redis calls, and Gemini evaluation calls.
- **CloudFront:** Global CDN and public URL. Caches/serves static files and forwards dynamic requests to Lambda.
- **S3:** Object storage for static build output such as JS/CSS/assets.
- **TiDB/MySQL:** Durable relational data: users, sessions, questions, attempts.
- **Upstash Redis:** Fast ephemeral layer: daily quota counters, evaluation cache, distributed locks.

Request Flow

```
User -> CloudFront
-> static asset? CloudFront/S3 serves it
-> dynamic route/API? CloudFront invokes Lambda
    -> Lambda runs Next.js route
    -> Prisma reads/writes TiDB/MySQL
    -> Redis handles quota/cache/lock
    -> Gemini is called only for fresh answer evaluation
-> response returns through CloudFront
```

Likely Follow-Ups

Why not EC2?

EC2 would work, but adds instance sizing, patching, process management, scaling rules, load balancing, and idle cost. Serverless fits because each request is mostly stateless.

What about cold starts?

Cold starts are the main tradeoff. For this interview/demo app, simplicity is worth it. If traffic grew, I would measure p95/p99 and consider provisioned concurrency for critical routes.

Where is state stored?

Lambda is disposable. Durable state is in TiDB/MySQL. Short-lived coordination is in Redis: quota, cache, and locks.

How are secrets handled?

SST passes server-side env vars into Lambda/build: DB URL, Redis URL/token, Gemini key. CI/CD supplies them from deployment secrets. They are not exposed to the browser.

Why CloudFront and S3?

S3 stores static files; CloudFront delivers them globally and caches them close to users. Dynamic requests are routed to Lambda.

What can bottleneck?

Downstream services: database queries/connections, Redis limits, Gemini latency/rate limits. Caching, quota, and locks reduce unnecessary load.

CI/CD And Short Memorized Version

GitHub Actions installs dependencies, builds the app, configures AWS credentials, and runs `npx sst deploy --stage production`. SST updates the AWS infrastructure and deploys the latest Next.js build.

Memorize: "SST is the deployment/IaC layer, CloudFront is the global entry point and CDN, S3 stores static Next.js assets, and Lambda runs the dynamic Next.js/API code. I chose serverless because the app is stateless per request, bursty, easy to scale horizontally, cheaper when idle, and easier to operate than EC2."